

This is a **backend-focused review of the event flow and invite system**. The issues below are not UX-level problems — they point to **state handling, session management, and logic inconsistencies** that need to be fixed at the system level.

1. Event Creation – Context & State Issues

Problem

- When creating an event from inside a pod, the system still asks to select the pod again.
- First attempt to create/schedule an event fails; second attempt succeeds.
- Users are getting “token expired” during active usage.

What this indicates

- Context (pod ID) is not being persisted or passed correctly.
- First request is failing silently or not committing state.
- Session/token handling is unstable (likely expiry or refresh logic issue).

Expected behavior

- Pod context should be pre-bound when creating an event inside a pod.
- Event creation should succeed on first request.
- Tokens should not expire during active interaction.

2. Invite Link Flow – Broken Redirection & State Sync

Problem

- Clicking invite link logs user in, but does not redirect to the event.
- User lands in a generic state and must manually navigate.
- In some cases, user gets “already a member” but is not admitted to the event.

What this indicates

- Invite link is not carrying or resolving event context correctly after auth.
- Post-login redirect is not mapped to event state.
- Membership check is blocking flow instead of resolving it.

Expected behavior

- Invite link → login (if needed) → automatic redirect into correct event.
- If user is already a member, they should be routed directly into the event, not blocked.

3. Invite Delivery & Identity Handling

Problem

- Invites are inconsistently delivered (some accounts receive, others do not).
- Magic link login is not always received.
- Same user across accounts behaves inconsistently.

What this indicates

- Email service or queue is unreliable or failing silently.
- Identity resolution between accounts is inconsistent.
- Possible mismatch between invite target and auth identity.

Expected behavior

- Invite delivery must be deterministic and logged.
- Magic links must always be delivered or fail with clear error.
- One identity = one consistent invite state.

4. Pending & Invalid Invite States

Problem

- Events that are finished still show pending invites.
- Deactivated users can still log in.
- Users can accept invites from notifications but not from direct links.

What this indicates

- No proper cleanup of invite state after event lifecycle ends.
- Deactivation is not enforced at auth/session level.
- Different entry points (notifications vs link) are not using the same validation logic.

Expected behavior

- Event end → all pending invites invalidated.
- Deactivated users cannot authenticate or access sessions.
- All invite entry points must follow the same validation logic.

5. Reminder System – Undefined Behavior

Problem

- “Remind” button behavior is unclear:
 - UI suggests bulk action, but unclear if it sends to one or all.
- Emails are not being sent as proper reminders.

What this indicates

- No clear mapping between UI action and backend trigger.
- Missing or incorrect email template handling.
- No distinction between single vs bulk reminder actions.

Expected behavior

- Clear API behavior:
 - Remind (single user)
 - Remind all (bulk action)
- Proper email template for reminder events.
- Logging of reminder sends.

6. Breakout Rooms & Matching – Core Logic Failure (Critical)

Problem

- Users:
 - Are assigned to multiple rooms.
 - Are matched with themselves.
 - Appear in rooms but are not visible to others.
- Matching issues:
 - Previously matched users are matched again.
 - Participant count is incorrect.
 - Room count changes dynamically without reason.
- Users must refresh to enter breakout rooms.

What this indicates

- Matching algorithm is not enforcing:
 - Unique user constraint
 - Unique pair constraint
- Session presence is not synchronized across clients.
- Room assignment is not atomic.
- Real-time updates (likely WebSocket/state sync) are failing.

Expected behavior

- One user → one room at a time (strict constraint).
- No self-matching.
- Previously matched pairs should not repeat (unless reset).
- Participant count must reflect real active users.
- Room assignment must be atomic and consistent across all clients.
- No manual refresh required — real-time state sync must work.

7. Session & Presence Management

Problem

- Users exist in multiple rooms simultaneously.
- Users are invisible to others despite being present.
- Leaving and rejoining does not update state correctly.

What this indicates

- Presence tracking is broken.
- Session state is duplicated or not cleaned up.
- Rejoin flow does not reconcile previous session.

Expected behavior

- Single active session per user per event.
- Proper cleanup on leave/disconnect.
- Rejoin should restore the correct state, not duplicate it.

8. Navigation & Flow Completion

Problem

- Users must refresh to:
 - Enter breakout rooms
 - Proceed to next stages
- After rating, users are not returned to main room.

What this indicates

- Frontend is not receiving or reacting to backend state transitions.
- Event lifecycle transitions are not pushed properly.

Expected behavior

- All transitions (match → room → rating → main room) should be event-driven.

- No manual refresh required at any step.

9. Invite List Handling (Scalability Issue)

Problem

- The invite list is not alphabetically sorted.
- Search results are inconsistent.

What this indicates

- Sorting is not enforced at query level.
- Search is not deterministic or properly indexed.

Expected behavior

- Alphabetical ordering by default.
- Predictable, indexed search behavior.

Summary – Core System Issues

The main problems are not isolated bugs but **system-level inconsistencies**:

- Event context is not reliably maintained.
- Invite flow breaks between steps (send → receive → login → join).
- Matching and room allocation are not enforcing core constraints.
- Session and presence state are not synchronized.
- Real-time updates are failing, forcing manual refresh.

Immediate Backend Priorities

1. Fix invite flow end-to-end with correct redirect and state resolution.
2. Stabilize matching logic with strict constraints (no duplicates, no self-match, correct counts).
3. Implement reliable session/presence management (single source of truth).
4. Fix real-time state sync (eliminate need for refresh).
5. Clean up event lifecycle handling (invites, rooms, transitions).

This needs to be addressed before any UI or design work, to make the current flow stable enough for immediate usage.